DOMAIN DRIVEN DESIGN USING JAVA · GLOBAL SOLUTION 2026/2

Mind Connect

Documentação técnica do back-end (API RESTful em Java com Spring Boot)

Tema: CitizenTech · FIAP

01. Capa e links

Integrantes

Nome	RM
Guilherme Joel de Camargo	RM570403
Pedro Felipe de Castro Rosa	RM569049
Pedro Henrique Cardoso Cavalcante De Souza	RM570464
Rafaela Donadio Figueiredo Tavares	RM572797
Victor Ferreira Gomes	RM569273

Links

Item	Endereço
API publicada no Render	COLE AQUI A URL DA API NO RENDER (ex.: https://mindconnect-api.onrender.com)
Vídeo de demonstração (YouTube não listado, até 5 min)	COLE AQUI O LINK DO VÍDEO

02. A solução

O problema do cidadão. Muitas pessoas não sabem onde encontrar serviços públicos e gratuitos de acolhimento em saúde mental (como CAPS e UBS), não têm acesso a informação confiável e não possuem um lugar simples para acompanhar como estão se sentindo. Essa falta de informação é uma barreira para quem precisa de apoio.

A solução. O **Mind Connect** é uma plataforma que centraliza informações sobre serviços de acolhimento, oferece uma biblioteca de conteúdos educativos e permite que cada pessoa mantenha, de forma voluntária e pessoal, um diário de bem-estar emocional. O módulo é exclusivamente informativo: não há cadastro de pacientes, agendamento de consultas nem encaminhamento médico, e a plataforma sempre incentiva a busca de atendimento profissional quando necessário.

Quem usa. Jovens e estudantes, pessoas que procuram informação sobre saúde mental, cidadãos que não conhecem os serviços públicos da sua região, familiares que querem ajudar alguém e os administradores que mantêm os conteúdos e os serviços atualizados.

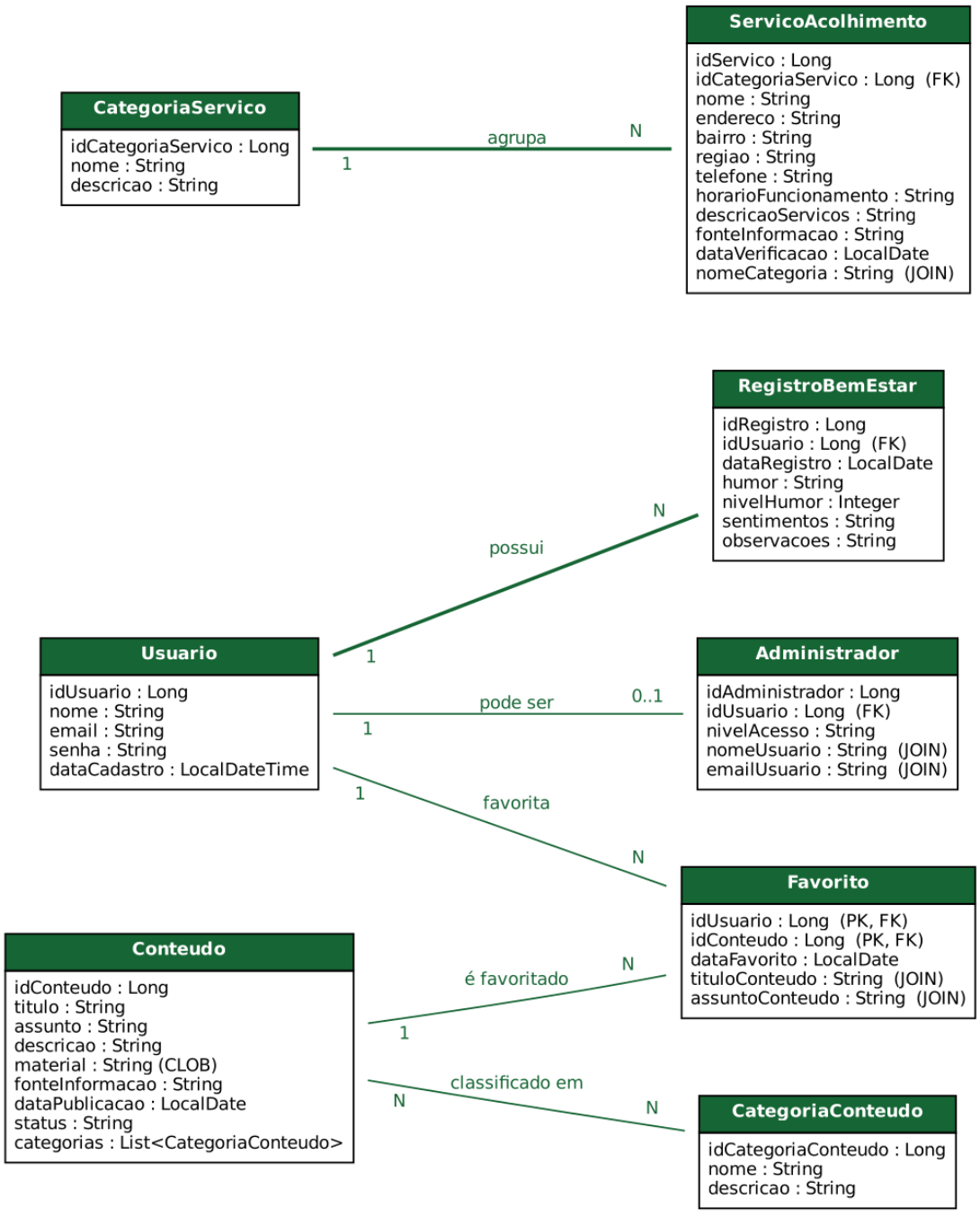
O que esta API resolve. Este projeto é o back-end da solução. Ele guarda os dados no Oracle da FIAP e expõe uma API REST consumida pelo front-end do grupo:

Módulo	O que a API faz
Usuários e login	Cadastro, edição, consulta e remoção de contas; login com senha protegida por hash.
Serviços de acolhimento	Diretório de unidades organizadas por categoria (CAPS, UBS...), com busca por nome e filtros por região e categoria.
Conteúdos educativos	Biblioteca de artigos classificados por categorias (ansiedade, estresse, sono...), com busca por assunto.
Diário de bem-estar	Registro pessoal de humor (nível de 1 a 5), com histórico por usuário.
Administração	Controle de quem tem acesso administrativo (ADMIN ou SUPERVISOR).

Alinhamento com CitizenTech. A tecnologia é usada para reduzir barreiras de acesso à informação e aos serviços públicos, aproximando o cidadão dos recursos disponíveis na sua comunidade. Os dados de serviços e conteúdos da carga inicial são demonstrativos (marcados como fictícios para teste acadêmico) e devem ser substituídos por fontes públicas verificadas antes de uso real.

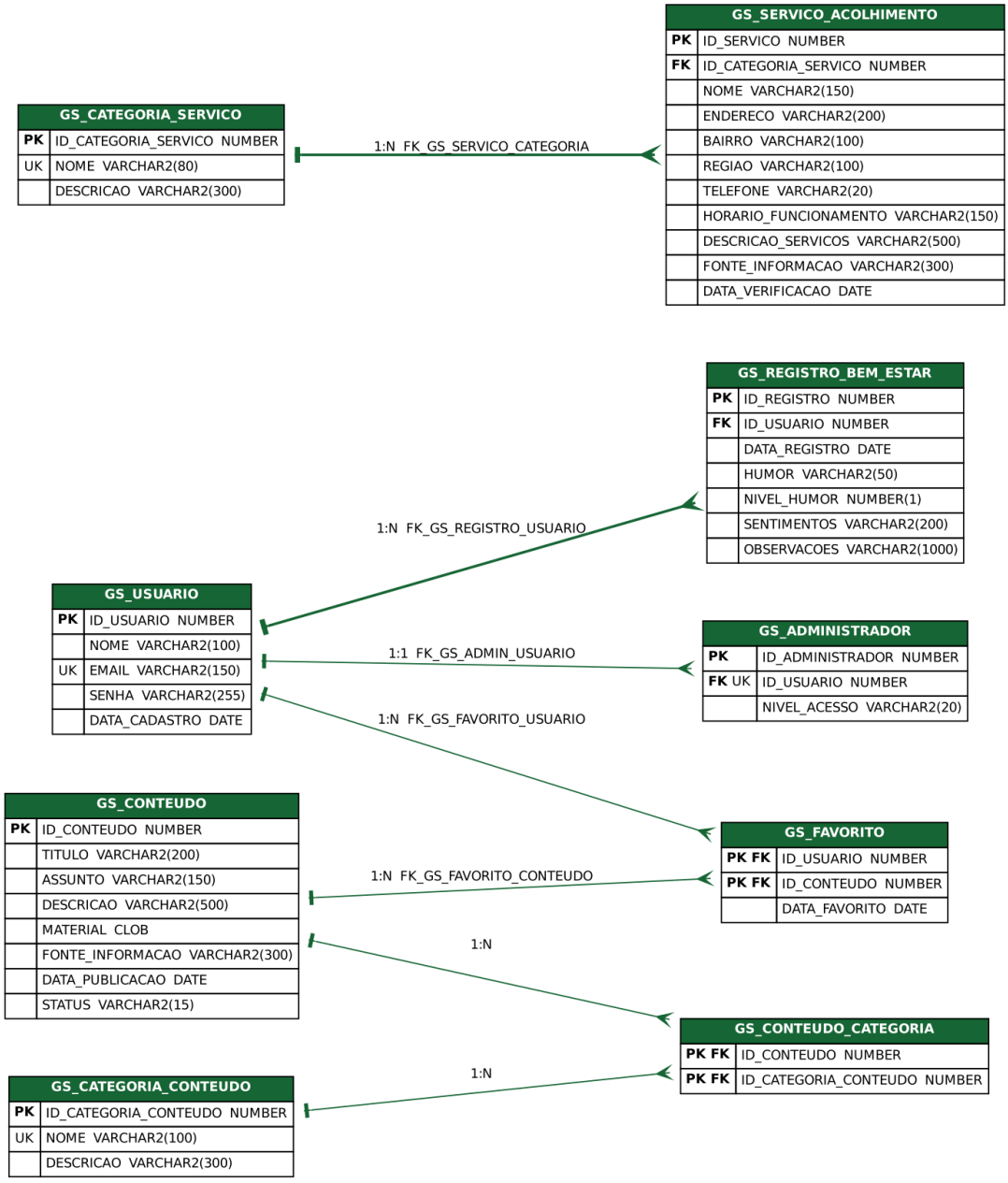
03. Diagrama de classes

Os models representam as tabelas do banco. Os relacionamentos **1:N** exigidos estão destacados em linha grossa: **CategoriaServico 1:N ServicoAcolhimento** e **Usuario 1:N RegistroBemEstar**. **Favorito** liga Usuario e Conteudo (chave composta). Os atributos marcados com (JOIN) vêm de consultas com junção e não são colunas da própria tabela.



04. Modelo do banco de dados (DER)

Banco Oracle da FIAP, com 9 tabelas (script `GSsqlDataBase.sql`, que começa com os DROP TABLE e depois recria tudo). As chaves primárias usam colunas `NUMBER GENERATED BY DEFAULT AS IDENTITY`. As chaves estrangeiras dos relacionamentos 1:N são **FK_GS_SERVICO_CATEGORIA** (GS_SERVICO_ACOLHIMENTO.ID_CATEGORIA_SERVICO → GS_CATEGORIA_SERVICO) e **FK_GS_REGISTRO_USUARIO** (GS_REGISTRO_BEM_ESTAR.ID_USUARIO → GS_USUARIO). A tabela **GS_FAVORITO** liga usuários a conteúdos (chave primária composta ID_USUARIO + ID_CONTEUDO, com FK para as duas tabelas).



Restrições: e-mail único em GS_USUARIO; nome único nas duas tabelas de categoria; um usuário só pode ter uma linha em GS_ADMINISTRADOR; o mesmo conteúdo não se repete nos favoritos de um usuário (chave composta); CHECK de NIVEL_HUMOR (1 a 5), STATUS do conteúdo (ATIVO/INATIVO) e NIVEL_ACESSO (ADMIN/SUPERVISOR).

05. Endpoints

Todas as rotas começam com `/api` e usam JSON. Padrão de status: **201** com header `Location` no cadastro, **200** nas consultas e atualizações, **204** na remoção, **400** dado inválido, **401** sem login válido ou login incorreto, **403** sem permissão, **404** não encontrado, **405** método não suportado, **409** regra de negócio violada ou remoção de registro com filhos e **500** erro inesperado (mensagem genérica). As rotas protegidas (veja o controle de acesso abaixo) também podem devolver **401** (sem token válido) e **403** (sem permissão), além dos erros da coluna.

Usuários (rotas aninhadas: diário de bem-estar do 1:N e conteúdos favoritos)

Método	Rota	Sucesso	Erros esperados
POST	/api/usuarios	201 + Location	400 · 409
GET	/api/usuarios	200	—
GET	/api/usuarios/{id}	200	404
PUT	/api/usuarios/{id}	200	400 · 404 · 409
DELETE	/api/usuarios/{id}	204	404 · 409
GET	/api/usuarios/{id}/bem-estar	200	404
POST	/api/usuarios/{id}/bem-estar	201 + Location	400 · 404 · 409
GET	/api/usuarios/{id}/favoritos	200	404
POST	/api/usuarios/{id}/favoritos	201 + Location	400 · 404 · 409
GET	/api/usuarios/{id}/favoritos/{idConteudo}	200	404
DELETE	/api/usuarios/{id}/favoritos/{idConteudo}	204	404
POST	/api/auth/login	200	400 · 401

Categorias de serviço (e rota aninhada do 1:N com os serviços)

Método	Rota	Sucesso	Erros esperados
POST	/api/categorias-servico	201 + Location	400 · 409
GET	/api/categorias-servico	200	—
GET	/api/categorias-servico/{id}	200	404
PUT	/api/categorias-servico/{id}	200	400 · 404 · 409
DELETE	/api/categorias-servico/{id}	204	404 · 409
GET	/api/categorias-servico/{id}/servicos	200	404
POST	/api/categorias-servico/{id}/servicos	201 + Location	400 · 404 · 409

Serviços de acolhimento

Método	Rota	Sucesso	Erros esperados
POST	/api/servicos	201 + Location	400 · 409
GET	/api/servicos	200	—
GET	/api/servicos/{id}	200	404
PUT	/api/servicos/{id}	200	400 · 404 · 409
DELETE	/api/servicos/{id}	204	404
GET	/api/servicos?nome=&regiao=&categoria=	200	—
GET	/api/servicos/regiao/{regiao}	200	—
GET	/api/servicos/categoria/{categoria}	200	—

Categorias de conteúdo

Método	Rota	Sucesso	Erros esperados
POST	/api/categorias-conteudo	201 + Location	400 · 409
GET	/api/categorias-conteudo	200	—
GET	/api/categorias-conteudo/{id}	200	404
PUT	/api/categorias-conteudo/{id}	200	400 · 404 · 409
DELETE	/api/categorias-conteudo/{id}	204	404 · 409

Conteúdos educativos

Método	Rota	Sucesso	Erros esperados
POST	/api/conteudos	201 + Location	400 · 409
GET	/api/conteudos	200	—
GET	/api/conteudos/{id}	200	404
PUT	/api/conteudos/{id}	200	400 · 404 · 409
DELETE	/api/conteudos/{id}	204	404
GET	/api/conteudos?busca=&status=&categoria=	200	—
GET	/api/conteudos/categoria/{categoria}	200	—

Diário de bem-estar

Método	Rota	Sucesso	Erros esperados
POST	/api/bem-estar	201 + Location	400 · 409
GET	/api/bem-estar	200	—
GET	/api/bem-estar/{id}	200	404
PUT	/api/bem-estar/{id}	200	400 · 404 · 409
DELETE	/api/bem-estar/{id}	204	404
GET	/api/bem-estar?idUsuario={id}	200	404
GET	/api/bem-estar/historico?idUsuario={id}	200	400 · 404

Administradores

Método	Rota	Sucesso	Erros esperados
POST	/api/administradores	201 + Location	400 · 409
GET	/api/administradores	200	—
GET	/api/administradores/{id}	200	404
PUT	/api/administradores/{id}	200	400 · 404 · 409
DELETE	/api/administradores/{id}	204	404

Notas: nos filtros, *categoria* aceita o id (número) ou o nome (ex.: CAPS, ANSIEDADE). Cadastro com id de referência inexistente no corpo (ex.: idCategoriaServico) devolve 400; na rota aninhada, o pai inexistente na URL devolve 404. Favorito não tem PUT (só guarda a data em que foi adicionado): usa POST para favoritar e DELETE para desfavoritar. Uma rota que não existe devolve 404 e um método não suportado devolve 405, ambos no JSON padronizado de erro.

Controle de acesso (permissões)

O login devolve um **token assinado** (HMAC-SHA256, válido por 8 horas). As rotas protegidas exigem o header `Authorization: Bearer <token>`. O id do usuário logado sai do token (que não pode ser alterado sem invalidar a assinatura), nunca de um dado que o cliente informe. A permissão de administrador é conferida no banco a cada requisição. Um interceptor confere tudo antes de o controller executar; rota sem classificação é tratada como exclusiva de administradores.

Nível	Quem pode	Rotas
Público	Qualquer pessoa	POST /api/usuarios (cadastro) · POST /api/auth/login · GET de serviços, categorias de serviço, conteúdos e categorias de conteúdo
Administrador	Somente administradores	POST/PUT/DELETE de /api/servicos, /api/categorias-servico (e POST /api/categorias-servico/{id}/servicos), /api/conteudos e /api/categorias-conteudo · todas as rotas de /api/administradores · GET /api/usuarios (lista de contas)
Proprietário	Somente o dono dos dados (o {id} da URL deve ser o do usuário logado; nem o administrador acessa)	/api/usuarios/{id}/bem-estar · /api/usuarios/{id}/favoritos (e /{idConteudo}) · PUT /api/usuarios/{id}
Proprietário ou administrador	O dono da conta ou um administrador	GET e DELETE /api/usuarios/{id}
Logado (dono do registro)	Usuário autenticado; cada um só vê e altera os próprios registros	/api/bem-estar: lista só os registros do logado; ?idUsuario= de outra pessoa → 403; POST com idUsuario de outra pessoa → 403; GET/PUT/DELETE /api/bem-estar/{id} de registro alheio → 403

Respostas: **401** quando falta o token ou ele é inválido/expirado; **403** quando o usuário está logado mas não tem permissão para a operação ou tenta acessar dados de outra pessoa.

Exemplos de JSON (entrada e saída)

POST /api/usuarios → 201 Created (Location: /api/usuarios/11)

```
// entrada
{ "nome": "Maria Silva", "email": "maria@example.com", "senha": "senha123" }
// saída (a senha nunca é devolvida)
{ "idUsuario": 11, "nome": "Maria Silva", "email": "maria@example.com", "dataCadastro": "2026-11-09T14:30:00" }
```

POST /api/auth/login → 200 OK

```
// entrada
{ "email": "maria@example.com", "senha": "senha123" }
// saída
{ "idUsuario": 11, "nome": "Maria Silva", "email": "maria@example.com", "administrador": false, "nivelAcesso": null,
  "token": "11.1793000000.Q8x...assinatura" }
// nas rotas protegidas, enviar o header:  Authorization: Bearer 11.1793000000.Q8x...assinatura
```

POST /api/categorias-servico → 201 Created

```
// entrada
{ "nome": "CRAS", "descricao": "Centro de Referência de Assistência Social" }
// saída
{ "idCategoriaServico": 5, "nome": "CRAS", "descricao": "Centro de Referência de Assistência Social" }
```

POST /api/servicos → 201 Created (rota aninhada equivalente: POST /api/categorias-servico/1/servicos, sem idCategoriaServico no corpo)

```
// entrada
{ "idCategoriaServico": 1, "nome": "CAPS Zona Leste", "endereco": "Rua das Flores, 120", "bairro": "Tatuapé",
  "regiao": "ZONA LESTE", "telefone": "(11) 3000-1234", "horarioFuncionamento": "Segunda a sexta, 8h às 17h",
  "descricaoServicos": "Acolhimento psicossocial", "fonteInformacao": "Prefeitura de São Paulo",
  "dataVerificacao": "2026-11-01" }
// saída
{ "idServico": 13, "idCategoriaServico": 1, "nomeCategoria": "CAPS", "nome": "CAPS Zona Leste",
  "endereco": "Rua das Flores, 120", "bairro": "Tatuapé", "regiao": "ZONA LESTE", "telefone": "(11) 3000-1234",
  "horarioFuncionamento": "Segunda a sexta, 8h às 17h", "descricaoServicos": "Acolhimento psicossocial",
  "fonteInformacao": "Prefeitura de São Paulo", "dataVerificacao": "2026-11-01" }
```

POST /api/categorias-conteudo → 201 Created

```
// entrada
{ "nome": "ADOLESCENCIA", "descricao": "Saúde mental na adolescência." }
// saída
{ "idCategoriaConteudo": 6, "nome": "ADOLESCENCIA", "descricao": "Saúde mental na adolescência." }
```

POST /api/conteudos → 201 Created

```
// entrada (status e dataPublicacao são opcionais)
{ "titulo": "Como lidar com a ansiedade", "assunto": "ANSIEDADE", "descricao": "Dicas gerais e educativas.",
  "material": "Texto completo do material educativo...", "fonteInformacao": "Ministério da Saúde",
  "dataPublicacao": "2026-11-09", "status": "ATIVO", "idsCategorias": [1, 3] }
// saída
{ "idConteudo": 11, "titulo": "Como lidar com a ansiedade", "assunto": "ANSIEDADE", "descricao": "Dicas gerais e educativas.",
  "material": "Texto completo do material educativo...", "fonteInformacao": "Ministério da Saúde",
  "dataPublicacao": "2026-11-09", "status": "ATIVO",
  "categorias": [ { "idCategoriaConteudo": 1, "nome": "ANSIEDADE", "descricao": "Materiais educativos sobre ansiedade." },
                  { "idCategoriaConteudo": 3, "nome": "ESTRESSE", "descricao": "Materiais educativos sobre estresse." } ] }
```

POST /api/bem-estar → 201 Created (rota aninhada equivalente: POST /api/usuarios/6/bem-estar, sem idUsuario no corpo)

```
// entrada (dataRegistro é opcional; padrão: hoje)
{ "idUsuario": 6, "dataRegistro": "2026-11-09", "humor": "BEM", "nivelHumor": 4,
  "sentimentos": "Tranquilidade", "observacoes": "Dia produtivo" }
// saída
{ "idRegistro": 7, "idUsuario": 6, "dataRegistro": "2026-11-09", "humor": "BEM", "nivelHumor": 4,
  "sentimentos": "Tranquilidade", "observacoes": "Dia produtivo" }
```

POST /api/administradores → 201 Created

```
// entrada (nivelAcesso é opcional; padrão: ADMIN)
{ "idUsuario": 6, "nivelAcesso": "SUPERVISOR" }
// saída
{ "idAdministrador": 4, "idUsuario": 6, "nivelAcesso": "SUPERVISOR", "nomeUsuario": "Felipe Rocha",
  "emailUsuario": "felipe.rocha@example.com" }
```

POST /api/usuarios/1/favoritos → 201 Created (Location: /api/usuarios/1/favoritos/5)

```
// entrada
{ "idConteudo": 5 }
// saída
{ "idUsuario": 1, "idConteudo": 5, "dataFavorito": "2026-11-09", "tituloConteudo": "Quando procurar acolhimento",
  "assuntoConteudo": "BUSCA DE AJUDA" }
```

PUT /api/usuarios/11 → 200 OK (a senha é opcional: sem ela, a atual é mantida)

```
// entrada
{ "nome": "Maria Silva Souza", "email": "maria@example.com" }
// saída
{ "idUsuario": 11, "nome": "Maria Silva Souza", "email": "maria@example.com", "dataCadastro": "2026-11-09T14:30:00" }
```

Formato padronizado de erro (todos os status de erro)

```
// 400 - validação (campos com problema em "campos")
{ "timestamp": "2026-11-09T14:35:10", "status": 400, "erro": "Bad Request",
  "mensagem": "Dados inválidos. Verifique os campos informados.", "caminho": "/api/usuarios",
  "campos": { "email": "E-mail inválido", "senha": "A senha deve ter entre 6 e 72 caracteres" } }

// 401 - rota protegida sem token válido
{ "timestamp": "2026-11-09T14:35:40", "status": 401, "erro": "Unauthorized",
  "mensagem": "Autenticação necessária. Faça login e envie o header Authorization: Bearer <token>.",
  "caminho": "/api/bem-estar", "campos": {} }

// 403 - sem permissão (ex.: usuário comum tentando cadastrar um serviço ou ler o diário de outra pessoa)
{ "timestamp": "2026-11-09T14:35:55", "status": 403, "erro": "Forbidden",
  "mensagem": "Esta operação é exclusiva de administradores.", "caminho": "/api/servicos", "campos": {} }

// 409 - regra de negócio
{ "timestamp": "2026-11-09T14:36:02", "status": 409, "erro": "Conflict",
  "mensagem": "Já existe um usuário cadastrado com o e-mail maria@example.com",
  "caminho": "/api/usuarios", "campos": {} }

// 500 - erro inesperado (mensagem genérica; o detalhe fica só no log do servidor)
{ "timestamp": "2026-11-09T14:40:00", "status": 500, "erro": "Internal Server Error",
  "mensagem": "Erro interno no servidor. Tente novamente mais tarde.", "caminho": "/api/servicos", "campos": {} }
```

06. Estrutura do projeto

O código fica em `br.com.mindconnect`, com um pacote por responsabilidade:

Pacote	Responsabilidade	Classes principais
model	Entidades do domínio (espelham as tabelas do Oracle).	Usuario, CategoriaServico, ServicoAcolhimento, CategoriaConteudo, Conteudo, RegistroBemEstar, Administrador, Favorito
dao	Acesso ao banco com JDBC puro: um DAO por entidade, **DataSource recebido pelo construtor** e PreparedStatement. Sem JPA, Hibernate ou Spring Data.	UsuarioDAO, CategoriaServicoDAO, ServicoAcolhimentoDAO, CategoriaConteudoDAO, ConteudoDAO, RegistroBemEstarDAO, AdministradorDAO, FavoritoDAO, TradutorErroSql
dto	Objetos de entrada e saída da API, com as validações do Bean Validation. A API nunca recebe nem devolve o model direto.	*EntradaDto, *AtualizarDto, *AninhadoDto, *SaidaDto, LoginEntradaDto, LoginSaidaDto, ErroDto
controller	Rotas REST, uso do @Valid, conversão com ModelMapper e status HTTP corretos (inclui o header Location).	UsuarioController, AuthController, CategoriaServicoController, ServicoAcolhimentoController, CategoriaConteudoController, ConteudoController, RegistroBemEstarController, AdministradorController
service	Regras de negócio entre o controller e o DAO (unicidade, um registro por dia, remoção de registro com filhos, hash de senha).	UsuarioService, AuthService, SenhaService, CategoriaServicoService, ServicoAcolhimentoService, CategoriaConteudoService, ConteudoService, RegistroBemEstarService, AdministradorService, FavoritoService, TokenService, AutorizacaoService, AdministradorInicialService
exception	Exceções próprias do domínio, cada uma ligada a um status HTTP.	RecursoNaoEncontradoException, RegraNegocioException, EmailJaCadastradoException, RegistroBemEstarDuplicadoException, RemocaoNaoPermitidaException, RequisicaoInvalidaException, CredenciaisInvalidasException, NaoAutenticadoException, AcessoNegadoException, ErroBancoDadosException
handler	@RestControllerAdvice que transforma **todo** erro em um JSON padronizado, sem stack trace nem detalhes de SQL.	GlobalExceptionHandler
config	Configurações: CORS liberado para o front, bean do ModelMapper e a segurança por perfil (interceptor + anotação @Acesso).	CorsConfig, ModelMapperConfig, SegurancaConfig, AutorizacaoInterceptor (confere token e permissão antes do controller), Acesso e Nivel (anotação que classifica cada rota), AdministradorInicialRunner

Fluxo de uma requisição: **Front-end** → **(HTTP + JSON, com CORS)** → **Controller** → **Service** → **DAO** → **(JDBC / DataSource)** → **Oracle FIAP**.

Arquivos da raiz: `pom.xml`, `Dockerfile` (multi-stage), `GSsqlDataBase.sql` (tabelas e carga de dados), `README.md` e `src/main/resources/application.properties` (usa apenas variáveis de ambiente).

07. Regras e validações

Regras de negócio

#	Regra	Onde	Resposta
RN1	O e-mail de cada usuário é único (sem diferenciar maiúsculas/minúsculas).	UsuarioService	409 · EmailJaCadastradoException
RN2	Cada usuário pode ter **apenas um registro de bem-estar por dia**.	RegistroBemEstarService	409 · RegistroBemEstarDuplicadoException
RN3	Não pode haver dois serviços com o mesmo nome e endereço.	ServicoAcolhimentoService	409 · RegraNegocioException
RN4	Registro com filhos não pode ser removido: categoria de serviço com serviços; usuário com registros de bem-estar, favoritos ou acesso de administrador; categoria de conteúdo com conteúdos associados. (Ao excluir um conteúdo, os favoritos dele saem junto.)	Services de Usuario, CategoriaServico e CategoriaConteudo	409 · RemocaoNaoPermitidaException
RN5	O nome das categorias (serviço e conteúdo) é único.	Services de categoria	409 · RegraNegocioException
RN6	Um usuário só pode ser administrador uma vez; o usuário precisa existir.	AdministradorService	409 / 400
RN7	Referência a registro pai inexistente: 400 quando o id vem no corpo; 404 quando vem na URL (rota aninhada).	Services	400 · 404
RN8	Senhas são guardadas com hash PBKDF2 + salt e nunca são devolvidas pela API. Login incorreto devolve a mesma mensagem para e-mail ou senha errados.	SenhaService / AuthService	401 · CredenciaisInvalidasException
RN9	Um usuário não pode favoritar o mesmo conteúdo duas vezes; o usuário (404) e o conteúdo (400) precisam existir.	FavoritoService	409 · RegraNegocioException
RN10	Controle de permissões: usuário comum consulta o catálogo, usa o diário e favorita conteúdos; só o administrador cadastra, altera e exclui serviços, conteúdos, categorias e administradores.	AutorizacaoInterceptor + @Acesso	403 · AcessoNegadoException
RN11	Cada usuário só consulta, altera ou exclui os próprios registros de bem-estar e favoritos; o dono é identificado pelo token, não pelo id enviado.	Interceptor e RegistroBemEstarService	403 · AcessoNegadoException
RN12	Rotas protegidas exigem token válido (assinado e dentro do prazo).	TokenService / AutorizacaoService	401 · NaoAutenticadoException
RN13	Valores padrão: conteúdo novo = status ATIVO e data de hoje; registro de bem-estar novo = data de hoje; administrador novo = nível ADMIN.	Services	—

Validações dos DTOs de entrada (Bean Validation, ativada com @Valid no controller)

DTO de entrada	Campo	Validações
UsuarioEntradaDto	nome	@NotBlank, @Size(max = 100)
	email	@NotBlank, @Email, @Size(max = 150)
	senha	@NotBlank, @Size(min = 6, max = 72)
UsuarioAtualizarDto	nome / email	iguais ao cadastro
	senha	opcional; se enviada, @Size(min = 6, max = 72)
LoginEntradaDto	email / senha	@NotBlank (+ @Email no e-mail)
CategoriaServicoEntradaDto	nome	@NotBlank, @Size(max = 80)
	descricao	@Size(max = 300)
ServicoAcolhimentoEntradaDto (e AninhadoDto, sem a categoria)	idCategoriaServico	@NotNull, @Positive
	nome	@NotBlank, @Size(max = 150)
	endereco	@NotBlank, @Size(max = 200)
	bairro	@Size(max = 100)
	regiao	@NotBlank, @Size(max = 100)
	telefone	@Size(max = 20), @Pattern (somente números, espaços, parênteses, + e -)
	horarioFuncionamento	@Size(max = 150)
	descricaoServicos	@Size(max = 500)
	fonteInformacao	@NotBlank, @Size(max = 300)
	dataVerificacao	@PastOrPresent
CategoriaConteudoEntradaDto	nome	@NotBlank, @Size(max = 100)
	descricao	@Size(max = 300)
ConteudoEntradaDto	titulo	@NotBlank, @Size(max = 200)
	assunto	@NotBlank, @Size(max = 150)
	descricao	@Size(max = 500)
	material	@NotBlank, @Size(max = 20000)
	fonteInformacao	@NotBlank, @Size(max = 300)
	status	@Pattern(ATIVO ou INATIVO); opcional
	idsCategorias	@Size(max = 10); cada id deve existir
RegistroBemEstarEntradaDto (e AninhadoDto, sem o usuário)	idUsuario	@NotNull, @Positive
	dataRegistro	@PastOrPresent; opcional
	humor	@NotBlank, @Size(max = 50)
	nivelHumor	@NotNull, @Min(1), @Max(5)
	sentimentos	@Size(max = 200)
	observacoes	@Size(max = 1000)
AdministradorEntradaDto	idUsuario	@NotNull, @Positive
	nivelAcesso	@Pattern(ADMIN ou SUPERVISOR); opcional
AdministradorAtualizarDto	nivelAcesso	@NotBlank, @Pattern(ADMIN ou SUPERVISOR)
FavoritoEntradaDto	idConteudo	@NotNull, @Positive

Além do Bean Validation, o handler global devolve 400 para JSON malformado ou com tipo errado (ex.: data fora do formato aaaa-mm-dd) e para ids da URL que não são números.

08. Deploy

A API é empacotada com um **Dockerfile multi-stage** (etapa 1: `maven:3.9-eclipse-temurin-21` compila o .jar; etapa 2: `eclipse-temurin:21-jre-alpine` executa só o .jar) e publicada no **Render** como *Web Service* com ambiente Docker, a partir do repositório do projeto. O Render informa a porta pela variável PORT, que a aplicação lê em `server.port`.

Passos de publicação

1. Subir o projeto (sem a pasta target) em um repositório Git.
2. No Render: *New* → *Web Service*, escolher o repositório e o ambiente *Docker*.
3. Cadastrar as variáveis de ambiente abaixo em *Environment*.
4. Fazer o deploy e testar com `GET /api/categorias-servico`.
5. Registrar a URL pública no README e nesta documentação.

Variáveis de ambiente (nomes, sem os valores)

Variável	Obrigatória	Para que serve
DB_URL	Sim	URL JDBC do Oracle da FIAP.
DB_USERNAME	Sim	Usuário do banco.
DB_PASSWORD	Sim	Senha do banco.
CORS_ORIGINS	Não	Endereços do front aceitos pelo CORS, separados por vírgula. Sem ela, qualquer origem é aceita.
AUTH_SECRET	Recomendada	Segredo (mínimo 16 caracteres) usado para assinar os tokens de login. Sem ela, é criado um segredo temporário e os logins caem quando a aplicação reinicia.
ADMIN_EMAIL	Recomendada	E-mail do administrador inicial (cria a conta ou ativa uma existente, como as da carga do SQL).
ADMIN_SENHA	Recomendada	Senha do administrador inicial (mínimo 6 caracteres). Os administradores da carga do SQL têm senha de demonstração e não conseguem logar sem isso.
PORT	Automática	Porta do servidor, definida pelo próprio Render.

Nenhum usuário, senha ou URL de banco fica no código entregue: o `application.properties` só referencia as variáveis. Sem as de banco a aplicação não inicia, de propósito. O Dockerfile também define o fuso horário America/Sao_Paulo.

Como rodar localmente

```
# 1) criar as tabelas e a carga de dados: executar GSsqlDataBase.sql no Oracle da FIAP
# 2) definir as variáveis de ambiente (Windows PowerShell)
$env:DB_URL = "jdbc:oracle:thin:@HOST:1521:SID"
$env:DB_USERNAME = "seu_usuario"
$env:DB_PASSWORD = "sua_senha"
$env:AUTH_SECRET = "um-segredo-longo-qualquer"
$env:ADMIN_EMAIL = "ana.souza@example.com"   # um usuário da carga (já é admin) ou um e-mail novo
$env:ADMIN_SENHA = "senha-do-admin"
# 3) rodar (Java 21 e Maven)
mvn spring-boot:run
# API em http://localhost:8080/api/...
```